

- #295.** 8.9 We can ensure that this string is valid by counting the number of left and right parens. It is always valid to add a left paren, up until the total number of pairs of parens. We can add a right paren as long as `count(left_parens) <= count(right_parens)`.
- #296.** 6.4 You can think about this either as the probability(3 ants walking clockwise) + probability(3 ants walking counter-clockwise). Or, you can think about it as: The first ant picks a direction. What's the probability of the other ants picking the same direction?
- #297.** 5.2 Think about what happens for values that can't be represented accurately in binary.
- #298.** 10.3 Can you modify binary search for this purpose?
- #299.** 11.1 What will happen to the unsigned `int`?
- #300.** 8.11 Try breaking it down into subproblems. If you were making change, what is the first choice you would make?
- #301.** 10.10 The problem with using an array is that it will be slow to insert a number. What other data structures could we use?
- #302.** 5.5 If `(n & (n-1)) == 0`, then this means that `n` and `n - 1` never have a 1 in the same spot. Why would that happen?
- #303.** 10.9 Another way to think about this is that if you drew a rectangle around a cell extending to the bottom, right coordinate of the matrix, the cell would be bigger than all the items in this square.
- #304.** 9.2 Is there any way to search from both the source and destination? For what reason or in what case might this be faster?
- #305.** 8.14 If your code looks really lengthy, with a lot of if's (for each possible operator, "target" boolean result, and left/right side), think about the relationship between the different parts. Try to simplify your code. It should not need a ton of complicated if-statements. For example, consider expressions of the form `<LEFT>OR<RIGHT>` versus `<LEFT>AND<RIGHT>`. Both may need to know the number of ways that the `<LEFT>` evaluates to true. See what code you can reuse.
- #306.** 6.9 The number 3 has an even number of factors (1 and 3). The number 12 has an even number of factors (1, 2, 3, 4, 6, 12). What numbers do not? What does this tell you about the doors?
- #307.** 7.12 Think carefully about what information the linked list node needs to contain.
- #308.** 8.12 We know that each row must have a queen. Can you try all possibilities?
- #309.** 8.7 Approach 2: To generate a permutation of `abcd`, you need to pick an initial character. It can be `a`, `b`, `c`, or `d`. You can then permute the remaining characters. How can you use this approach to generate all permutations of the full string?
- #310.** 10.3 What is the runtime of your algorithm? What will happen if the array has duplicates?
- #311.** 9.5 How would you scale this to a larger system?
- #312.** 5.4 Get Next: Can you flip a 0 to a 1 to create the next biggest number?
- #313.** 11.4 Think about what load testing is designed to test. What are the factors in the load of a webpage? What criteria would be used to judge if a webpage performs satisfactorily under heavy load?

- #314.** 5.3 Each sequence can be lengthened by merging it with an adjacent sequence (if any) or just flipping the immediate neighboring zero. You just need to find the best choice.
- #315.** 10.8 Consider implementing your own bit vector class. It's a good exercise and an important part of this problem.
- #316.** 10.11 You should be able to design an $O(n)$ algorithm.
- #317.** 10.9 A cell will be larger than all the items below it and to the right. It will be smaller than all cells above it and to the left. If we wanted to eliminate the most elements first, which element should we compare the value x to?
- #318.** 8.6 If you're having trouble with recursion, then try trusting the recursive process more. Once you've figured out how to move the top two disks from tower 0 to tower 2, trust that you have this working. When you need to move three disks, trust that you can move two disks from one tower to another. Now, two disks have been moved. What do you do about the third?
- #319.** 6.1 Imagine there were just three bottles and one had heavier pills. Suppose you put different numbers of pills from each bottle on the scale (for example, bottle 1 has 5 pills, bottle 2 has 2 pills, and bottle 3 has 9 pills). What would the scale show?
- #320.** 10.4 Think about how binary search works. What will be the issue with just implementing binary search?
- #321.** 9.2 Discuss how you might implement these algorithms and this system in the real world. What sort of optimizations might you make?
- #322.** 8.13 Once we pick the box on the bottom, we need to pick the second box. Then the third box.
- #323.** 6.2 The probability of making two out of three shots is $\text{probability}(\text{make shot 1, make shot 2, miss shot 3}) + \text{probability}(\text{make shot 1, miss shot 2, make shot 3}) + \text{probability}(\text{miss shot 1, make shot 2, make shot 3}) + \text{probability}(\text{make shot 1, make shot 2, make shot 3})$.
- #324.** 8.11 If you were making change, the first choice you might make is how many quarters you need to use.
- #325.** 11.2 Think about issues both within the program and outside of the program (the rest of the system).
- #326.** 9.4 Estimate how much space is needed for this.
- #327.** 8.14 Look at your recursion. Do you have repeated calls anywhere? Can you memoize it?
- #328.** 5.7 The value 1010 in binary is 10 in decimal or 0xA in hex. What will a sequence of 101010... be in hex? That is, how do you represent an alternating sequence of 1s and 0s with 1s in the odd places? How do you do this for the reverse (1s in the even spots)?
- #329.** 11.3 Consider both extreme cases and more general cases.
- #330.** 10.9 If we compare x to the center element in the matrix, we can eliminate roughly one quarter of the elements in the matrix.
- #331.** 8.2 For the robot to reach the last cell, it must find a path to the second-to-last cells. For it to find a path to the second-to-last cells, it must find a path to the third-to-last cells.
- #332.** 10.1 Try moving from the end of the array to the beginning.

- #333.** 6.8 If we drop Egg 1 at fixed intervals (e.g., every 10 floors), then the worst case is the worst case for Egg 1 + the worst case for Egg 2. The problem with our earlier solutions is that as Egg 1 does more work, Egg 2 doesn't do any less work. Ideally, we'd like to balance this a bit. As Egg 1 does more work (has survived more drops), Egg 2 should have less work to do. What might this mean?
- #334.** 9.3 Think about how infinite loops might occur.
- #335.** 8.7 Approach 2: To generate all permutations of `abcd`, pick each character (`a`, `b`, `c`, or `d`) as a starting character. Permute the remaining characters and prepend the starting character. How do you permute the remaining characters? With a recursive process that follows the same logic.
- #336.** 5.6 How would you figure out how many bits are different between two numbers?
- #337.** 10.4 Binary search requires comparing an element to the midpoint. Getting the midpoint requires knowing the length. We don't know the length. Can we find it?
- #338.** 8.4 Subsets that contain `c` will be subsets `{a, b, c}` but not `{a, b}`. Can you build these subsets from the subsets of `{a, b}`?
- #339.** 5.4 Get Next: Flipping a 0 to a 1 will create a bigger number. The farther right the index is the smaller the bigger number is. If we have a number like 1001, we want to flip the rightmost 0 (to create 1011). But if we have a number like 1010, we should not flip the rightmost 1.
- #340.** 8.3 Given a specific index and value, can you identify if the magic index would be before or after it?
- #341.** 6.6 Now suppose there were two blue-eyed people. What would they see? What would they know? When would they leave? Remember your answer from the prior hint. Assume they know the answer to the earlier hint.
- #342.** 10.2 Do you even need to truly "sort"? Or is just reorganizing the list sufficient?
- #343.** 8.11 Once you've decided to use two quarters to make change for 98 cents, you now need to figure out how many ways to make change for 48 cents using nickels, dimes, and pennies.
- #344.** 7.5 Think about all the different functionality a system to read books online would have to support. You don't have to do everything, but you should think about making your assumptions explicit.
- #345.** 11.4 Could you build your own? What might that look like?
- #346.** 5.5 What is the relationship between how `n` looks and how `n - 1` looks? Walk through a binary subtraction.
- #347.** 9.4 Will you need multiple passes? Multiple machines?
- #348.** 10.4 We can find the length by using an exponential backoff. First check index 2, then 4, then 8, then 16, and so on. What will be the runtime of this algorithm?
- #349.** 11.6 What can we automate?
- #350.** 8.12 Each row must have a queen. Start with the last row. There are eight different columns on which you can put a queen. Can you try each of these?

- #351. 7.10 Should number cells, blank cells, and bomb cells be separate classes?
- #352. 5.3 Try to do it in linear time, a single pass, and $O(1)$ space.
- #353. 9.3 How would you detect the same page? What does this mean?
- #354. 8.4 You can build the remaining subsets by adding c to all the subsets of $\{a, b\}$.
- #355. 5.7 Try masks $0xaaaaaaaa$ and $0x55555555$ to select the even and odd bits. Then try shifting the even and odd bits around to create the right number.
- #356. 8.7 Approach 2: You can implement this approach by having the recursive function pass back the list of the strings, and then you prepend the starting character to it. Or, you can push down a prefix to the recursive calls.
- #357. 6.8 Try dropping Egg 1 at bigger intervals at the beginning and then at smaller and smaller intervals. The idea is to keep the sum of Egg 1 and Egg 2's drops as constant as possible. For each additional drop that Egg 1 takes, Egg 2 takes one fewer drop. What is the right interval?
- #358. 5.4 Get Next: We should flip the rightmost non-trailing 0. The number 1010 would become 1110. Once we've done that, we need to flip a 1 to a 0 to make the number as small as possible, but bigger than the original number (1010). What do we do? How can we shrink the number?
- #359. 8.1 Try memoization as a way to optimize an inefficient recursive program.
- #360. 8.2 Simplify this problem a bit by first figuring out if there's a path. Then, modify your algorithm to track the path.
- #361. 7.10 What is the algorithm to place the bombs around the board?
- #362. 11.1 Look at the parameters for `printf`.
- #363. 7.2 Before coding, make a list of the objects you need and walk through the common algorithms. Picture the code. Do you have everything you need?
- #364. 8.10 Think about this as a graph.
- #365. 9.3 How do you define if two pages are the same? Is it the URLs? Is it the content? Both of these can be flawed. Why?
- #366. 5.8 First try the naive approach. Can you set a particular "pixel"?
- #367. 6.3 Picture a domino laying down on the board. How many black squares does it cover? How many white squares?
- #368. 8.13 Once you have a basic recursive algorithm implemented, think about if you can optimize it. Are there any repeated subproblems?
- #369. 5.6 Think about what an XOR indicates. If you do a $XOR\ b$, where does the result have 1s? Where does it have 0s?
- #370. 6.6 Build up from this. What if there were three blue-eyed people? What if there were four blue-eyed people?
- #371. 8.12 Break this down into smaller subproblems. The queen at row 8 must be at column 1, 2, 3, 4, 5, 6, 7, or 8. Can you print all ways of placing eight queens where a queen is at row 8 and column 3? You then need to check all the ways of placing a queen on row 7.

- #372.** 5.5 When you do a binary subtraction, you flip the rightmost 0s to a 1, stopping when you get to a 1 (which is also flipped). Everything (all the 1s and 0s) on the left will stay put.
- #373.** 8.4 You can also do this by mapping each subset to a binary number. The i th bit could represent a “boolean” flag for whether an element is in the set.
- #374.** 6.8 Let X be the first drop of Egg 1. This means that Egg 2 would do $X - 1$ drops if Egg 1 broke. We want to try to keep the sum of Egg 1 and Egg 2’s drops as constant as possible. If Egg 1 breaks on the second drop, then we want Egg 2 to do $X - 2$ drops. If Egg 1 breaks on the third drop, then we want Egg 2 to do $X - 3$ drops. This keeps the sum of Egg 1 and Egg 2 fairly constant. What is X ?
- #375.** 5.4 Get Next: We can shrink the number by moving all the 1s to the right of the flipped bit as far right as possible (removing a 1 in the process).
- #376.** 10.10 Would it work well to use a binary search tree?
- #377.** 7.10 To place the bombs randomly on the board: Think about the algorithm to shuffle a deck of cards. Can you apply a similar technique?
- #378.** 8.13 Alternatively, we can think about the repeated choices as: Does the first box go on the stack? Does the second box go on the stack? And so on.
- #379.** 6.5 If you fill the 5-quart jug and then use it to fill the 3-quart jug, you’ll have two quarts left in the 5-quart jug. You can either keep those two quarts where they are, or you can dump the contents of the smaller jug and pour the two quarts in there.
- #380.** 8.11 Analyze your algorithm. Is there any repeated work? Can you optimize this?
- #381.** 5.8 When you’re drawing a long line, you’ll have entire bytes that will become a sequence of 1s. Can you set this all at once?
- #382.** 8.10 You can implement this using depth-first search (or breadth-first search). Each adjacent pixel of the “right” color is a connected edge.
- #383.** 5.5 Picture n and $n - 1$. To subtract 1 from n , you flipped the rightmost 1 to a 0 and all the 0s on its right to 1s. If $n \& n - 1 == 0$, then there are no 1s to the left of the first 1. What does that mean about n ?
- #384.** 5.8 What about the start and end of the line? Do you need to set those pixels individually, or can you set them all at once?
- #385.** 9.1 Think about this as a real-world application. What are the different factors you would need to consider?
- #386.** 7.10 How do you count the number of bombs neighboring a cell? Will you iterate through all cells?
- #387.** 6.1 You should be able to have an equation that tells you the heavy bottle based on the weight.
- #388.** 8.2 Think again about the efficiency of your algorithm. Can you optimize it?
- #389.** 7.9 The `rotate()` method should be able to run in $O(1)$ time.
- #390.** 5.4 Get Previous: Once you’ve solved Get Next, try to invert the logic for Get Previous.
- #391.** 5.8 Does your code handle the case when x_1 and x_2 are in the same byte?
- #392.** 10.10 Consider a binary search tree where each node stores some additional data.

- #393. 11.6 Have you thought about security and reliability?
- #394. 8.11 Try using memoization.
- #395. 6.8 I got 14 drops in the worst case. What did you get?
- #396. 9.1 There's no one right answer here. Discuss several different technical implementations.
- #397. 6.3 How many black squares are there on the board? How many white squares?
- #398. 5.5 We know that n must have only one 1 if $n \& (n-1) == 0$. What sorts of numbers have only one 1?
- #399. 7.10 When you click on a blank cell, what is the algorithm to expand the neighboring cells?
- #400. 6.5 Once you've developed a way to solve this problem, think about it more broadly. If you are given a jug of size X and another jug of size Y , can you always use it to measure Z ?
- #401. 11.3 Is it possible to test everything? How will you prioritize testing?



Hints for Knowledge-Based Questions

- #402. 12.9 Focus on the concept firsts, then worry about the exact implementation. How should `SmartPointer` look?
- #403. 15.2 A context switch is the time spent switching between two processes. This happens when you bring one process into execution and swap out the existing process.
- #404. 13.1 Think about who can access private methods.
- #405. 15.1 How do these differ in terms of memory?
- #406. 12.11 Recall that a two dimensional array is essentially an array of arrays.
- #407. 15.2 Ideally, we would like to record the timestamp when one process "stops" and the timestamp when another process "starts." But how do we know when this swapping will occur?
- #408. 14.1 A `GROUP BY` clause might be useful.
- #409. 13.2 When does a `finally` block get executed? Are there any cases where it won't get executed?
- #410. 12.2 Can we do this in place?
- #411. 14.2 It might be helpful to break the approach into two pieces. The first piece is to get each building ID and the number of open requests. Then, we can get the building names.
- #412. 13.3 Consider that some of these might have different meanings depending on where they are applied.
- #413. 12.10 Typically, `malloc` will just give us an arbitrary block of memory. If we can't override this behavior, can we work with it to do what we need?
- #414. 15.7 First implement the single-threaded FizzBuzz problem.
- #415. 15.2 Try setting up two processes and have them pass a small amount of data back and forth. This will encourage the system to stop one process and bring the other one in.
- #416. 13.4 The purpose of these might be somewhat similar, but how does the implementation differ?
- #417. 15.5 How can we ensure that `first()` has terminated before calling `second()`?
- #418. 12.11 One approach is to call `malloc` for each array. How would we free the memory here?
- #419. 15.3 A deadlock can happen when there's a "cycle" in the order of who is waiting for whom. How can we break or prevent this cycle?

- #420. 13.5 Think about the underlying data structure.
- #421. 12.7 Think about why we use virtual methods.
- #422. 15.4 If every thread had to declare upfront what processes it might need, could we detect possible deadlocks in advance?
- #423. 12.3 What is the underlying data structure behind each? What are the implications of this?
- #424. 13.5 HashMap uses an array of linked lists. TreeMap uses a red-black tree. LinkedHashMap uses doubly-linked buckets. What is the implication of this?
- #425. 13.4 Consider the usage of primitive types. How else might they differ in terms of how you can use the types?
- #426. 12.11 Can we allocate this instead as a contiguous block of memory?
- #427. 12.8 This data structure can be pictured as a binary tree, but it's not necessarily. What if there's a loop in the structure?
- #428. 14.7 You probably need a list of students, their courses, and another table building a relationship between students and courses. Note that this is a many-to-many relationship.
- #429. 15.6 The keyword `synchronized` ensures that two threads cannot execute synchronized methods on the same instance at the same time.
- #430. 13.5 Consider how they might differ in terms of the order of iteration through the keys. Why might you want one option instead of the others?
- #431. 14.3 First try to get a list of the IDs (just the IDs) of all the relevant apartments.
- #432. 12.10 Imagine we have a sequential set of integers (3, 4, 5, ...). How big does this set need to be to ensure that one of the numbers is divisible by 16?
- #433. 15.5 Why would using `boolean` flags to do this be a bad idea?
- #434. 15.4 Think about the order of requests as a graph. What does a deadlock look like within this graph?
- #435. 13.6 Object reflection allows you to get information about methods and fields in an object. Why might this be useful?
- #436. 14.6 Be particularly careful about which relationships are one-to-one vs. one-to-many vs. many-to-many.
- #437. 15.3 One idea is to just not let a philosopher hold onto a chopstick if he can't get the other one.
- #438. 12.9 Think about tracking the number of references. What will this tell us?
- #439. 15.7 Don't try to do anything fancy on the single-threaded problem. Just get something that is simple and easily readable.
- #440. 12.10 How will we free the memory?
- #441. 15.2 It's okay if your solution isn't totally perfect. That might not be possible. Discuss the tradeoffs of your approach.
- #442. 14.7 Think carefully about how you handle ties when selecting the top 10%.

- #443.** 13.8 A naive approach is to pick a random subset size z and then iterate through the elements, putting it in the set with probability $z/\text{list_size}$. Why would this not work?
- #444.** 14.5 Denormalization means adding redundant data to a table. It's typically used in very large systems. Why might this be useful?
- #445.** 12.5 A shallow copy copies just the initial data structure. A deep copy does this, and also copies any underlying data. Given this, why might you use one versus the other?
- #446.** 15.5 Would semaphores be useful here?
- #447.** 15.7 Outline the structure for the threads without worrying about synchronizing anything.
- #448.** 13.7 Consider how you'd implement this first without lambda expressions.
- #449.** 12.1 If we already had the number of lines in the file, how would we do this?
- #450.** 13.8 Pick the list of all the subsets of an n -element set. For any given item x , half of the subsets contain x and half do not.
- #451.** 14.4 Describe INNER JOINS and OUTER JOINS. OUTER JOINS can have multiple types: left, right, and full.
- #452.** 12.2 Be careful about the null character.
- #453.** 12.9 What are all the different methods/operators we might want to override?
- #454.** 13.5 What would the runtime of the common operations be?
- #455.** 14.5 Think about the cost of joins on a large system.
- #456.** 12.6 The keyword `volatile` signals that a variable might be changed from outside of the program, such as by another process. Why might this be necessary?
- #457.** 13.8 Do not pick the length of the subset in advance. You don't need to. Instead, think about this as picking whether each element will be put into the set.
- #458.** 15.7 Once you get the structure of each thread done, think about what you need to synchronize.
- #459.** 12.1 Suppose we didn't have the number of lines in the file. Is there a way we could do this without first counting the number of lines?
- #460.** 12.7 What would happen if the destructor were not virtual?
- #461.** 13.7 Break this up into two parts: filtering the countries and then getting a sum.
- #462.** 12.8 Consider using a hash table.
- #463.** 12.4 You should discuss vtables here.
- #464.** 13.7 Can you do this without a filter operation?

IV

Hints for Additional Review Problems

- #465. 16.3 Think about what you're going to design for.
- #466. 16.12 Consider a recursive or tree-like approach.
- #467. 17.1 Walk through binary addition by hand (slowly!) and try to really understand what is happening.
- #468. 16.13 Draw a square and a bunch of lines that cut it in half. Where are those lines located?
- #469. 17.24 Start with a brute force solution.
- #470. 17.14 There are actually several approaches. Brainstorm these. It's okay to start off with a naive approach.
- #471. 16.20 Consider recursion.
- #472. 16.3 Will all lines intercept? What determines if two lines intercept?
- #473. 16.7 Let k be 1 if $a > b$ and 0 otherwise. If you were given k , could you return the max (without a comparison or if-else logic)?
- #474. 16.22 The tricky bit is handling an infinite grid. What are your options?
- #475. 17.15 Try simplifying this problem: What if you just needed to know the longest word made up of two other words in the list?
- #476. 16.10 Solution 1: Can you count the number of people alive in each year?
- #477. 17.25 Start by grouping the dictionary by the word lengths, since you know each column has to be the same length and each row has to be the same length.
- #478. 17.7 Discuss the naive approach: merging names together when they are synonyms. How would you identify transitive relationships? $A == B$, $A == C$, and $C == D$ implies $A == D == B == C$.
- #479. 16.13 Any straight line that cuts a square in half goes through the center of the square. How then can you find a line that cuts two squares in half?
- #480. 17.17 Start with a brute force solution. What is the runtime?
- #481. 16.22 Option #1: Do you actually need an infinite grid? Read the problem again. Do you know the max size of the grid?
- #482. 16.16 Would it help to know the longest sorted sequences at the beginning and end?
- #483. 17.2 Try approaching this problem recursively.

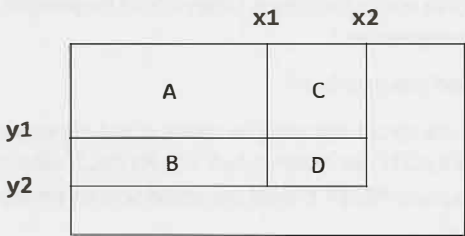
- #484.** 17.26 Solution 1: Start with just a simple algorithm comparing all documents to all other documents. How would you compute the similarity of two documents as fast as possible?
- #485.** 17.5 It doesn't really matter which letter or number it is. You can simplify this problem to just having an array of As and Bs. You would then be looking for the longest subarray with an equal number of As and Bs.
- #486.** 17.11 Consider first the algorithm for finding the closest distance if you will run the algorithm only once. You should be able to do this in $O(N)$ time, where N is the number of words in the document.
- #487.** 16.20 Can you recursively try all possibilities?
- #488.** 17.9 Be clear about what this problem is asking for. It's asking for the k th smallest number in the form $3^a * 5^b * 7^c$.
- #489.** 16.2 Think about what the best conceivable runtime is for this problem. If your solution matches the best conceivable runtime, then you probably can't do any better.
- #490.** 16.10 Solution 1: Try using a hash table, or an array that maps from a birth year to how many people are alive in that year.
- #491.** 16.14 Sometimes, a brute force is a pretty good solution. Can you try all possible lines?
- #492.** 16.1 Try picturing the two numbers, a and b , on a number line.
- #493.** 17.7 The core part of the problem is to group names into the various spellings. From there, figuring out the frequencies is relatively easy.
- #494.** 17.3 If you haven't already, solve 17.2 on page 186.
- #495.** 17.16 There are recursive and iterative solutions to this problem, but it's probably easier to start with the recursive solution.
- #496.** 17.13 Try a recursive approach.
- #497.** 16.3 Infinite lines will almost always intersect—unless they're parallel. Parallel lines might still "intersect"—if they're the same lines. What does this mean for line segments?
- #498.** 17.26 Solution 1: To compute the similarity of two documents, try reorganizing the data in some way. Sorting? Using another data structure?
- #499.** 17.15 If we wanted to know just the longest word made up of other words in the list, then we could iterate over all words, from longest to shortest, checking if each could be made up of other words. To check this, we split the string in all possible locations.
- #500.** 17.25 Can you find a word rectangle of a specific length and width? What if you just tried all options?
- #501.** 17.11 Adapt your algorithm for one execution of the algorithm for repeated executions. What is the slow part? Can you optimize it?
- #502.** 16.8 Try thinking about the number in terms of chunks of three digits.
- #503.** 17.19 Start with the first part: Finding the missing number if only one number is missing.

- #504.** 17.16 Recursive solution: You have two choices at each appointment (take the appointment or reject the appointment). As a brute force approach, you can recurse through all possibilities. Note, though, that if you take request i , your recursive algorithm should skip request $i + 1$.
- #505.** 16.23 Be very careful that your solution actually returns each value from 0 through 6 with equal probability.
- #506.** 17.22 Start with a brute force, recursive solution. Just create all words that are one edit away, check if they are in the dictionary, and then attempt that path.
- #507.** 16.10 Solution 2: What if you sorted the years? What would you sort by?
- #508.** 17.9 What does a brute force solution to get the k th smallest value for $3^a * 5^b * 7^c$ look like?
- #509.** 17.12 Try a recursive approach.
- #510.** 17.26 Solution 1: You should be able to get an $O(A+B)$ algorithm to compute the similarity of two documents.
- #511.** 17.24 The brute force solution requires us to continuously compute the sums of each matrix. Can we optimize this?
- #512.** 17.7 One thing to try is maintaining a mapping of each name to its “true” spelling. You would also need to map from a true spelling to all the synonyms. Sometimes, you might need to merge two different groups of names. Play around with this algorithm to see if you can get it to work. Then see if you can simplify/optimize it.
- #513.** 16.7 If k were 1 when $a > b$ and 0 otherwise, then you could return $a * k + b * (\text{not } k)$. But how do you create k ?
- #514.** 16.10 Solution 2: Do you actually need to match the birth years and death years? Does it matter when a specific person died, or do you just need a list of the years of deaths?
- #515.** 17.5 Start with a brute force solution.
- #516.** 17.16 Recursive solution: You can optimize this approach through memoization. What is the runtime of this approach?
- #517.** 16.3 How can we find the intersection between two lines? If two line segments intercept, then this must be at the same point as their “infinite” extensions. Is this intersection point within both lines?
- #518.** 17.26 Solution 1: What is the relationship between the intersection and the union? Can you compute one from the other?
- #519.** 17.20 Recall that the median means the number for which half the numbers are larger and half the numbers are smaller.
- #520.** 16.14 You can’t truly try all possible lines in the world—that’s infinite. But you know that a “best” line must intersect at least two points. Can you connect each pair of points? Can you check if each line is indeed the best line?
- #521.** 16.26 Can we just process the expression from left to right? Why might this fail?
- #522.** 17.10 Start with a brute force solution. Can you just check each value to see if it’s the majority element?

- #523.** 16.10 Solution 2: Observe that people are “fungible.” It doesn’t matter who was born and when they died. All you need is a list of birth years and death years. This might make the question of how you sort the list of people easier.
- #524.** 16.25 First scope the problem. What are the features you would want?
- #525.** 17.24 Can you do any sort of precomputation to make computing the sum of a submatrix $O(1)$?
- #526.** 17.16 Recursive solution: The runtime of your memoization approach should be $O(N)$, with $O(N)$ space.
- #527.** 16.3 Think carefully about how to handle the case of line segments that have the same slope and y-intercept.
- #528.** 16.13 To cut two squares in half, a line must go through the middle of both squares.
- #529.** 16.14 You should be able to get to an $O(N^2)$ solution.
- #530.** 17.14 Consider thinking about reorganizing the data in some way or using additional data structures.
- #531.** 16.17 Picture the array as alternating sequences of positive and negative numbers. Observe that we would never include just part of a positive sequence or part of a negative sequence.
- #532.** 16.10 Solution 2: Try creating a sorted list of births and a sorted list of deaths. Can you iterate through both, tracking the number of people alive at any one time?
- #533.** 16.22 Option #2: Think about how an `ArrayList` works. Can you use an `ArrayList` for this?
- #534.** 17.26 Solution 1: To understand the relationship between the union and the intersection of two sets, consider a Venn diagram (a diagram where one circle overlaps another circle).
- #535.** 17.22 Once you have a brute force solution, try to find a faster way of getting all valid words that are one edit away. You don’t want to create all strings that are one edit away when the vast majority of them are not valid dictionary words.
- #536.** 16.2 Can you use a hash table to optimize the repeated case?
- #537.** 17.7 An easier way of taking the above approach is to have each name map to a list of alternate spellings. What should happen when a name in one group is set equal to a name in another group?
- #538.** 17.11 You could build a lookup table that maps from a word to a list of the locations where each word appears. How then could you find the closest two locations?
- #539.** 17.24 What if you precomputed the sum of the submatrix starting at the top left corner and continuing to each cell? How long would it take you to compute this? If you did this, could you then get the sum of an arbitrary submatrix in $O(1)$ time?
- #540.** 16.22 Option #2: It’s not impossible to use an `ArrayList`, but it would be tedious. Perhaps it would be easier to build your own, but specialized for matrices.
- #541.** 16.10 Solution 3: Each birth adds one person and each death removes a person. Try writing an example of a list of people (with birth and death years) and then re-formatting this into a list of each year and a +1 for a birth and a -1 for a death.

- #542.** 17.16 Iterative solution: Take the recursive solution and investigate it more. Can you implement a similar strategy iteratively?
- #543.** 17.15 Extend the earlier idea to multiple words. Can we just break each word up in all possible ways?
- #544.** 17.1 You can think about binary addition as iterating through the number, bit by bit, adding two bits, and then carrying over the one if necessary. You could also think about it as grouping the operations. What if you first added each of the bits (without carrying any overflow)? After that, you can handle the overflow.
- #545.** 16.21 Do some math here or play around with some examples. What does this pair need to look like? What can you say about their values?
- #546.** 17.20 Note that you have to store all the elements you've seen. Even the smallest of the first 100 elements could become the median. You can't just toss very low or very high elements.
- #547.** 17.26 Solution 2: It's tempting to try to think of minor optimizations—for example, keeping track of the min and max elements in each array. You could then figure out quickly, in specific cases, if two arrays don't overlap. The problem with that (and other optimizations along these lines) is that you still need to compare all documents to all other documents. It doesn't leverage the fact that the similarity is sparse. Given that we have a lot of documents, we really need to not compare all documents to all other documents (even if that comparison is very fast). All such solutions will be $O(D^2)$, where D is the number of documents. We shouldn't compare all documents to all other documents.
- #548.** 16.24 Start with a brute force solution. What is the runtime? What is the best conceivable runtime for this problem?
- #549.** 16.10 Solution 3: What if you created an array of years and how the population changed in each year? Could you then find the year with the highest population?
- #550.** 17.9 In looking for the k th smallest value of $3^a * 5^b * 7^c$, we know that a , b , and c will be less than or equal to k . Can you generate all such numbers?
- #551.** 16.17 Observe that if you have a sequence of values which have a negative sum, those will never start or end a sequence. (They could be present in a sequence if they connected two other sequences.)
- #552.** 17.14 Can you sort the numbers?
- #553.** 16.16 We can think about the array as divided into three subarrays: LEFT, MIDDLE, RIGHT. LEFT and RIGHT are both sorted. The MIDDLE elements are in an arbitrary order. We need to expand MIDDLE until we could sort those elements and then have the entire array sorted.
- #554.** 17.16 Iterative solution: It's probably easiest to start with the end of the array and work backwards.
- #555.** 17.26 Solution 2: If we can't compare all documents to all other documents, then we need to dive down and start looking at things at the element level. Consider a naive solution and see if you can extend that to multiple documents.

- #556. 17.22 To quickly get the valid words that are one edit away, try to group the words in the dictionary in a useful way. Observe that all words in the form `b_ll` (such as `bill`, `ball`, `bell`, and `bull`) will be one edit away. However, those aren't the only words that are one edit away from `bill`.
- #557. 16.21 When you move a value `a` from array `A` to array `B`, then `A`'s sum decreases by `a` and `B`'s sum increases by `a`. What happens when you swap two values? What would be needed to swap two values and get the same sum?
- #558. 17.11 If you had a list of the occurrences of each word, then you are really looking for a pair of values within two arrays (one value for each array) with the smallest difference. This could be a fairly similar algorithm to your initial algorithm.
- #559. 16.22 Option #2: One approach is to just double the size of the array when the ant wanders to an edge. How will you handle the ant wandering into negative coordinates, though? Arrays can't have negative indices.
- #560. 16.13 Given a line (slope and y-intercept), can you find where it intersects another line?
- #561. 17.26 Solution 2: One way to think about this is that we need to be able to very quickly pull a list of all documents with some similarity to a specific document. (Again, we should not do this by saying "look at all documents and quickly eliminate the dissimilar documents." That will be at least $O(D^2)$.)
- #562. 17.16 Iterative solution: Observe that you would never skip three appointments in a row. Why would you? You would always be able to take the middle booking.
- #563. 16.14 Have you tried using a hash table?
- #564. 16.21 If you swap two values, `a` and `b`, then the sum of `A` becomes `sumA - a + b` and the sum of `B` becomes `sumB - b + a`. These sums need to be equal.
- #565. 17.24 If you can precompute the sum from the top left corner to each cell, you can use this to compute the sum of an arbitrary submatrix in $O(1)$ time. Picture a particular submatrix. The full, precomputed sum will include this submatrix, an array immediately above it (`C`), and array to the left (`B`), and an area to the top and left (`A`). How can you compute the sum of just `D`?



- #566. 17.10 Consider the brute force solution. We pick an element and then validate if it's the majority element by counting the number of matching and non-matching elements. Suppose, for the first element, the first few checks reveal seven non-matching elements and three matching elements. Is it necessary to keep checking this element?
- #567. 16.17 Start from the beginning of the array. As that subsequence gets larger, it stays as the best subsequence. Once it becomes negative, though, it's useless.

- #568.** 17.16 Iterative solution: If you take appointment i , you will never take appointment $i + 1$, but you will always take appointment $i + 2$ or $i + 3$.
- #569.** 17.26 Solution 2: Building off the earlier hint, we can ask what defines the list of documents with some similarity to a document like {13, 16, 21, 3}. What attributes does that list have? How would we gather all documents like that?
- #570.** 16.22 Option #2: Observe that nothing in the problem stipulates that the label for the coordinates must remain the same. Can you move the ant and all cells into positive coordinates? In other words, what would happen if, whenever you needed to grow the array in a negative direction, you relabeled all the indices such that they were still positive?
- #571.** 16.21 You are looking for values a and b where $\text{sumA} - a + b = \text{sumB} - b + a$. Do the math to work out what this means for a and b 's values.
- #572.** 16.9 Approach these one by one, starting with subtraction. Once you've completed one function, you can use it to implement the others.
- #573.** 17.6 Start with a brute force solution.
- #574.** 16.23 Start with a brute force solution. How many times does it call `rand5()` in the worst case?
- #575.** 17.20 Another way to think about this is: Can you maintain the bottom half of elements and the top half of elements?
- #576.** 16.10 Solution 3: Be careful with the little details in this problem. Does your algorithm/code handle a person who dies in the same year that they are born? This person should be counted as one person in the population count.
- #577.** 17.26 Solution 2: The list of documents similar to {13, 16, 21, 3} includes all documents with a 13, 16, 21, and 3. How can we efficiently find this list? Remember that we'll be doing this for many documents, so some precomputing can make sense.
- #578.** 17.16 Iterative solution: Use an example and work backwards. You can easily find the optimal solution for the subarrays $\{r_n\}$, $\{r_{n-1}, r_n\}$, $\{r_{n-2}, \dots, r_n\}$. How would you use those to quickly find the optimal solution for $\{r_{n-3}, \dots, r_n\}$?
- #579.** 17.2 Suppose you had a method `shuffle` that worked on decks up to $n - 1$ elements. Could you use this method to implement a new `shuffle` method that works on decks up to n elements?
- #580.** 17.22 Create a mapping from a wildcard form (like `b_ll`) to all words in that form. Then, when you want to find all words that are one edit away from `bill`, you can look up `_ll`, `b_ll`, `bi_l`, and `bil_` in the mapping.
- #581.** 17.24 The sum of just D will be $\text{sum}(A \& B \& C \& D) - \text{sum}(A \& B) - \text{sum}(A \& C) + \text{sum}(A)$.
- #582.** 17.17 Can you use a trie?
- #583.** 16.21 If we do the math, we are looking for a pair of values such that $a - b = (\text{sumA} - \text{sumB}) / 2$. The problem then reduces to looking for a pair of values with a particular difference.
- #584.** 17.26 Solution 2: Try building a hash table from each word to the documents that contain this word. This will allow us to easily find all documents with some similarity to {13, 16, 21, 3}.
- #585.** 16.5 How does a zero get into the result of $n!$? What does it mean?

- #586.** 17.7 If each name maps to a list of its alternate spellings, you might have to update a lot of lists when you set X and Y as synonyms. If X is a synonym of {A, B, C}, and Y is a synonym of {D, E, F} then you would need to add {Y, D, E, F} to A's synonym list, B's synonym list, C's synonym list, and X's synonym list. Ditto for {Y, D, E, F}. Can we make this faster?
- #587.** 17.16 Iterative solution: If you take an appointment, you can't take the next appointment, but you can take anything after that. Therefore, $\text{optimal}(r_i, \dots, r_n) = \max(r_i + \text{optimal}(r_{i+2}, \dots, r_n), \text{optimal}(r_{i+1}, \dots, r_n))$. You can solve this iteratively by working backwards.
- #588.** 16.8 Have you considered negative numbers? Does your solution work for values like 100,030,000?
- #589.** 17.15 When you get recursive algorithms that are very inefficient, try looking for repeated subproblems.
- #590.** 17.19 Part 1: If you have to find the missing number in $O(1)$ space and $O(N)$ time, then you can do a only constant number of passes through the array and can store only a few variables.
- #591.** 17.9 Look at the list of all values for $3^a * 5^b * 7^c$. Observe that each value in the list will be $3 * (\text{some previous value})$, $5 * (\text{some previous value})$, or $7 * (\text{some previous value})$.
- #592.** 16.21 A brute force solution is to just look through all pairs of values to find one with the right difference. This will probably look like an outer loop through A with an inner loop through B. For each value, compute the difference and compare it to what we're looking for. Can we be more specific here, though? Given a value in A and a target difference, do we know the exact value of the element within B we're looking for?
- #593.** 17.14 What about using a heap or tree of some sort?
- #594.** 16.17 If we tracked the running sum, we should reset it as soon as the subsequence becomes negative. We would never add a negative sequence to the beginning or end of another subsequence.
- #595.** 17.24 With precomputation, you should be able to get a runtime of $O(N^4)$. Can you make this even faster?
- #596.** 17.3 Try this recursively. Suppose you had an algorithm to get a subset of size m from $n - 1$ elements. Could you develop an algorithm to get a subset of size m from n elements?
- #597.** 16.24 Can we make this faster with a hash table?
- #598.** 17.22 Your previous algorithm probably resembles a depth-first search. Can you make this faster?
- #599.** 16.22 Option #3: Another thing to think about is whether you even need a grid to implement this. What information do you actually need in the problem?
- #600.** 16.9 Subtraction: Would a negate function (which converts a positive integer to negative) help? Can you implement this using the add operator?
- #601.** 17.1 Focus on just one of the steps above. If you "forgot" to carry the ones, what would the add operation look like?

- #602.** 16.21 What the brute force really does is look for a value within B which equals $a - \text{target}$. How can you more quickly find this element? What approaches help us quickly find out if an element exists within an array?
- #603.** 17.26 Solution 2: Once you have a way of easily finding the documents similar to a particular document, you can go through and just compute the similarity to those documents using a simple algorithm. Can you make this faster? Specifically, can you compute the similarity directly from the hash table?
- #604.** 17.10 The majority element will not necessarily look like the majority element at first. It is possible, for example, to have the majority element appear in the first element of the array and then not appear again for the next eight elements. However, in those cases, the majority element will appear later in the array (in fact, many times later on in the array). It's not necessarily critical to continue checking a specific instance of an element for majority status once it's already looking "unlikely."
- #605.** 17.7 Instead, X, A, B, and C should map to the same instance of the set {X, A, B, C}. Y, D, E, and F should map to the same instance of {Y, D, E, F}. When we set X and Y as synonyms, we can then just copy one of the sets into the other (e.g., add {Y, D, E, F} to {X, A, B, C}). How else do we change the hash table?
- #606.** 16.21 We can use a hash table here. We can also try sorting. Both help us locate elements more quickly.
- #607.** 17.16 Iterative solution: If you're careful about what data you really need, you should be able to solve this in $O(n)$ time and $O(1)$ additional space.
- #608.** 17.12 Think about it this way: If you had methods called `convertLeft` and `convertRight` (which would convert left and right subtrees to doubly linked lists), could you put those together to convert the whole tree to a doubly linked list?
- #609.** 17.19 Part 1: What if you added up all the values in the array? Could you then figure out the missing number?
- #610.** 17.4 How long would it take you to figure out the least significant bit of the missing number?
- #611.** 17.26 Solution 2: Imagine you are looking up the documents similar to {1, 4, 6} by using a hash table that maps from a word to documents. The same document ID appears multiple times when doing this lookup. What does that indicate?
- #612.** 17.6 Rather than counting the number of twos in each number, think about digit by digit. That is, count the number of twos in the first digit (for each number), then the number of twos in the second digit (for each number), then the number of twos in the third digit (for each number), and so on.
- #613.** 16.9 Multiply: it's easy enough to implement `multiply` using `add`. But how do you handle negative numbers?
- #614.** 16.17 You can solve this in $O(N)$ time and $O(1)$ space.
- #615.** 17.24 Suppose this was just a single array. How could we compute the subarray with the largest sum? See 16.17 for a solution to this.
- #616.** 16.22 Option #3: All you actually need is some way of looking up if a cell is white or black (and of course the position of the ant). Can you just keep a list of all the white cells?

- #617.** 17.17 One solution is to insert every suffix of the larger string into the trie. For example, if the word is dogs, the suffixes would be dogs, ogs, gs, and s. How would this help you solve the problem? What is the runtime here?
- #618.** 17.22 A breadth-first search will often be faster than a depth-first search—not necessarily in the worst case, but in many cases. Why? Can you do something even faster than this?
- #619.** 17.5 What if you just started from the beginning, counting the number of As and the number of Bs you've seen so far? (Try making a table of the array and the number of As and Bs thus far.)
- #620.** 17.10 Note also that the majority element must be the majority element for some subarray and that no subarray can have multiple majority elements.
- #621.** 17.24 Suppose I just wanted you to find the maximum submatrix starting at row r_1 and ending at row r_2 , how could you most efficiently do this? (See the prior hint.) If I now wanted you find the maximum subarray from r_1 to (r_2+2) , could you do this efficiently?
- #622.** 17.9 Since each number is 3, 5, or 7 times a previous value in the list, we could just check all possible values and pick the next one that hasn't been seen yet. This will result in a lot of duplicated work. How can we avoid this?
- #623.** 17.13 Can you just try all possibilities? What might that look like?
- #624.** 16.26 Multiplication and division are higher priority operations. In an expression like $3*4 + 5*9/2 + 3$, the multiplication and division parts need to be grouped together.
- #625.** 17.14 If you picked an arbitrary element, how long would it take you to figure out the rank of this element (the number of elements bigger or smaller than it)?
- #626.** 17.19 Part 2: We're now looking for two missing numbers, which we will call a and b . The approach from part 1 will tell us the sum of a and b , but it won't actually tell us a and b . What other calculations could we do?
- #627.** 16.22 Option #3: You could consider keeping a hash set of all the white cells. How will you be able to print the whole grid, though?
- #628.** 17.1 The adding step alone would convert $1 + 1 \rightarrow 0$, $1 + 0 \rightarrow 1$, $0 + 1 \rightarrow 1$, $0 + 0 \rightarrow 0$. How do you do this without the $+$ sign?
- #629.** 17.21 What role does the tallest bar in the histogram play?
- #630.** 16.25 What data structure would be most useful for the lookups? What data structure would be most useful to know and maintain the order of items?
- #631.** 16.18 Start with a brute force approach. Can you try all possibilities for a and b ?
- #632.** 16.6 What if you sorted the arrays?
- #633.** 17.11 Can you just iterate through both arrays with two pointers? You should be able to do it in $O(A+B)$ time, where A and B are the sizes of the two arrays.
- #634.** 17.2 You could build this algorithm recursively by swapping the n th element for any of the elements before it. What would this look like iteratively?
- #635.** 16.21 What if the sum of A is 11 and the sum of B is 8? Can there be a pair with the right difference? Check that your solution handles this situation appropriately.

- #636.** 17.26 Solution 3: There's an alternative solution. Consider taking all of the words from all of the documents, throwing them into one giant list, and sorting this list. Assume you could still know which document each word came from. How could you track the similar pairs?
- #637.** 16.23 Make a table indicating how each possible sequence of calls to `rand5()` would map to the result of `rand7()`. For example, if you were implementing `rand3()` with `(rand2() + rand2()) % 3`, then the table would look like the below. Analyze this table. What can it tell you?
- | | 1st | 2nd | Result |
|--|-----|-----|--------|
| | 0 | 0 | 0 |
| | 0 | 1 | 1 |
| | 1 | 0 | 1 |
| | 1 | 1 | 2 |
- #638.** 17.8 This problem asks us to find the longest sequence of pairs you can build such that both sides of the pair are constantly increasing. What if you needed only one side of the pair to increase?
- #639.** 16.15 Try first creating an array with the frequency that each item occurs.
- #640.** 17.21 Picture the tallest bar, and then the next tallest bar on the left and the next tallest bar on the right. The water will fill the area between those. Can you calculate that area? What do you do about the rest?
- #641.** 17.6 Is there a faster way of calculating how many twos are in a particular digit across a range of numbers? Observe that roughly $\frac{1}{10}$ th of any digit should be a 2—but only roughly. How do you make that more exact?
- #642.** 17.1 You can do the add step with an XOR.
- #643.** 16.18 Observe that one of the substrings, either a or b, must start at the beginning of the string. That cuts down the number of possibilities.
- #644.** 16.24 What if the array were sorted?
- #645.** 17.18 Start with a brute force solution.
- #646.** 17.12 Once you have a basic idea for a recursive algorithm, you might get stuck on this: sometimes your recursive algorithm needs to return the start of the linked list, and sometimes it needs to return the end. There are multiple ways of solving this issue. Brainstorm some of them.
- #647.** 17.14 If you picked an arbitrary element, you would, on average, wind up with an element around the 50th percentile mark (half the elements above it and half the elements below). What if you did this repeatedly?
- #648.** 16.9 Divide: If you're trying to compute, where $x = \frac{a}{b}$, remember that $a = bx$. Can you find the closest value for x? Remember that this is integer division and x should be an integer.
- #649.** 17.19 Part 2: There are a lot of different calculations we could try. For example, we could multiply all the numbers, but that will only lead us to the product of a and b.
- #650.** 17.10 Try this: Given an element, start checking if this is the start of a subarray for which it's the majority element. Once it's become "unlikely" (appears less than half the time), start checking at the next element (the element after the subarray).

- #651.** 17.21 You can calculate the area between the tallest bar overall and the tallest bar on the left by just iterating through the histogram and subtracting out any bars in between. You can do the same thing with the right side. How do you handle the remainder of the graph?
- #652.** 17.18 One brute force solution is to take each starting position and move forward until you've found a subsequence which contains all the target characters.
- #653.** 16.18 Don't forget to handle the possibility that the first character in the pattern is b.
- #654.** 16.20 In the real world, we should know that some prefixes/substrings won't work. For example, consider the number 33835676368. Although 3383 does correspond to ffft, there are no words that start with ffft. Is there a way we can short-circuit in cases like this?
- #655.** 17.7 An alternative approach is to think of this as a graph. How would this work?
- #656.** 17.13 You can think about the choices the recursive algorithm makes in one of two ways: (1) At each character, should I put a space here? (2) Where should I put the next space? You can solve both of these recursively.
- #657.** 17.8 If you needed only one side of the pair to increase, then you would just sort all the values on that side. Your longest sequence would in fact be all of the pairs (other than any duplicates, since the longest sequence needs to strictly increase). What does this tell you about the original problem?
- #658.** 17.21 You can handle the remainder of the graph by just repeating this process: find the tallest bar and the second tallest bar, and subtract out the bars in between.
- #659.** 17.4 To find the least significant bit of the missing number, note that you know how many 0s and 1s to expect. For example, if you see three 0s and three 1s in the least significant bit, then the missing number's least significant bit must be a 1. Think about it: in any sequence of 0s and 1s, you'd get a 0, then a 1, then a 0, then a 1, and so on.
- #660.** 17.9 Rather than checking all values in the list for the next value (by multiplying each by 3, 5, and 7), think about it this way: when you insert a value x into the list, you can "create" the values 3x, 5x, and 7x to be used later.
- #661.** 17.14 Think about the previous hint some more, particularly in the context of quicksort.
- #662.** 17.21 How can you make the process of finding the next tallest bar on each side faster?
- #663.** 16.18 Be careful with how you analyze the runtime. If you iterate through $O(n^2)$ substrings and each one does an $O(n)$ string comparison, then the total runtime is $O(n^3)$.
- #664.** 17.1 Now focus on the carrying. In what cases will values carry? How do you apply the carry to the number?
- #665.** 16.26 Consider thinking about it as, when you get to a multiplication or division sign, jumping to a separate "process" to compute the result of this chunk.
- #666.** 17.8 If you sort the values based on height, then this will tell you the ordering of the final pairs. The longest sequence must be in this relative order (but not necessarily containing all of the pairs). You now just need to find the longest increasing subsequence on weight while keeping the items in the same relative order. This is essentially the same problem as having an array of integers and trying to find the longest sequence you can build (without reordering those items).

- #667.** 16.16 Consider the three subarrays: LEFT, MIDDLE, RIGHT. Focus on just this question: Can you sort middle such that the entire array becomes sorted? How would you check this?
- #668.** 16.23 Looking at this table again, note that the number of rows will be 5^k , where k is the max number of calls to `rand5()`. In order to make each value between 0 and 6 have equal probability, $\frac{1}{7}$ th of the rows must map to 0, $\frac{1}{7}$ th to 1, and so on. Is this possible?
- #669.** 17.18 Another way of thinking about the brute force is that we take each starting index and find the next instance of each element in the target string. The maximum of all these next instances marks the end of a subsequence which contains all the target characters. What is the runtime of this? How can we make it faster?
- #670.** 16.6 Think about how you would merge two sorted arrays.
- #671.** 17.5 When the above tables have equal values for the number of As and Bs, the entire subarray (starting from index 0) has an equal number of As and Bs. How could you use this table to find qualifying subarrays that don't start at index 0?
- #672.** 17.19 Part 2: Adding the numbers together will tell us the result of $a + b$. Multiplying the numbers together will tell us the result of $a * b$. How can we get the exact values for a and b ?
- #673.** 16.24 If we sorted the array, we could do repeated binary searches for the complement of a number. What if, instead, the array is given to us sorted? Could we then solve the problem in $O(N)$ time and $O(1)$ space?
- #674.** 16.19 If you were given the row and column of a water cell, how can you find all connected spaces?
- #675.** 17.7 We can treat adding X, Y as synonyms as adding an edge between the X node and the Y node. How then do we figure out the groups of synonyms?
- #676.** 17.21 Can you do precomputation to compute the next tallest bar on each side?
- #677.** 17.13 Will the recursive algorithm hit the same subproblems repeatedly? Can you optimize with a hash table?
- #678.** 17.14 What if, when you picked an element, you swapped elements around (as you do in quicksort) so that the elements below it would be located before the elements above it? If you did this repeatedly, could you find the smallest one million numbers?
- #679.** 16.6 Imagine you had the two arrays sorted and you were walking through them. If the pointer in the first array points to 3 and the pointer in the second array points to 9, what effect will moving the second pointer have on the difference of the pair?
- #680.** 17.12 To handle whether your recursive algorithm should return the start or the end of the linked list, you could try to pass a parameter down that acts as a flag. This won't work very well, though. The problem is that when you call `convert(current.left)`, you want to get the end of left's linked list. This way you can join the end of the linked list to current. But, if current is someone else's right subtree, `convert(current)` needs to pass back the start of the linked list (which is actually the start of current.left's linked list). Really, you need both the start and end of the linked list.
- #681.** 17.18 Consider the previously explained brute force solution. A bottleneck is repeatedly asking for the next instance of a particular character. Is there a way you can optimize this? You should be able to do this in $O(1)$ time.

- #682.** 17.8 Try a recursive approach that just evaluates all possibilities.
- #683.** 17.4 Once you've identified that the least significant bit is a 0 (or a 1), you can rule out all the numbers without 0 as the least significant bit. How is this problem different from the earlier part?
- #684.** 17.23 Start with a brute force solution. Can you try the biggest possible square first?
- #685.** 16.18 Suppose you decide on a specific value for the "a" part of a pattern. How many possibilities are there for b?
- #686.** 17.9 When you add x to the list of the first k values, you can add 3x, 5x, and 7x to some new list. How do you make this as optimal as possible? Would it make sense to keep multiple queues of values? Do you always need to insert 3x, 5x, and 7x? Or, perhaps sometimes you need to insert only 7x? You want to avoid seeing the same number twice.
- #687.** 16.19 Try recursion to count the number of water cells.
- #688.** 16.8 Consider dividing up a number into sequences of three digits.
- #689.** 17.19 Part 2: We could do both. If we know that $a + b = 87$ and $a * b = 962$, then we can solve for a and b: $a = 13$ and $b = 74$. But this will also result in having to multiply really large numbers. The product of all the numbers could be larger than 10^{157} . Is there a simpler calculation you can make?
- #690.** 16.11 Consider building a diving board. What are the choices you make?
- #691.** 17.18 Can you precompute the next instance of a particular character from each index? Try using a multi-dimensional array.
- #692.** 17.1 The carry will happen when you are doing $1 + 1$. How do you apply the carry to the number?
- #693.** 17.21 As an alternative solution, think about it from the perspective of each bar. Each bar will have water on top of it. How much water will be on top of each bar?
- #694.** 16.25 Both a hash table and a doubly linked list would be useful. Can you combine the two?
- #695.** 17.23 The biggest possible square is $N \times N$. So if you try that square first and it works, then you know that you've found the best square. Otherwise, you can try the next smallest square.
- #696.** 17.19 Part 2: Almost any "equation" we can come up with will work here (as long as it's not equivalent to a linear sum). It's just a matter of keeping this sum small.
- #697.** 16.23 It is not possible to divide 5^k evenly by 7. Does this mean that you can't implement `rand7()` with `rand5()`?
- #698.** 16.26 You can also maintain two stacks, one for the operators and one for the numbers. You push a number onto the stack every time you see it. What about the operators? When do you pop operators from the stack and apply them to the numbers?
- #699.** 17.8 Another way to think about the problem is this: if you had the longest sequence ending at each element $A[0]$ through $A[n-1]$, could you use that to find the longest sequence ending at element $A[n-1]$?
- #700.** 16.11 Consider a recursive solution.

- #701.** 17.12 Many people get stuck at this point and aren't sure what to do. Sometimes they need the start of the linked list, and sometimes they need the end. A given node doesn't necessarily know what to return on its `convert` call. Sometimes the simple solution is easiest: always return both. What are some ways you could do this?
- #702.** 17.19 Part 2: Try a sum of squares of the values.
- #703.** 16.20 A trie might help us short-circuit. What if you stored the whole list of words in the trie?
- #704.** 17.7 Each connected subgraph represents a group of synonyms. To find each group, we can do repeated breadth-first (or depth-first) searches.
- #705.** 17.23 Describe the runtime of the brute force solution.
- #706.** 16.19 How can you make sure that you're not revisiting the same cells? Think about how breadth-first search or depth-first search on a graph works.
- #707.** 16.7 When $a > b$, then $a - b > 0$. Can you get the sign bit of $a - b$?
- #708.** 16.16 In order to be able to sort `MIDDLE` and have the whole array become sorted, you need `MAX(LEFT) <= MIN(MIDDLE and RIGHT)` and `MAX(LEFT and MIDDLE) <= MIN(RIGHT)`.
- #709.** 17.20 What if you used a heap? Or two heaps?
- #710.** 16.4 If you were calling `hasWon` multiple times, how might your solution change?
- #711.** 16.5 Each zero in $n!$ corresponds to n being divisible by a factor of 10. What does that mean?
- #712.** 17.1 You can use an AND operation to compute the carry. What do you do with it?
- #713.** 17.5 Suppose, in this table, index i has `count(A, 0->i) = 3` and `count(B, 0->i) = 7`. This means that there are four more Bs than As. If you find a later spot j with the same difference (`count(B, 0->j) - count(A, 0->j)`), then this indicates a subarray with an equal number of As and Bs.
- #714.** 17.23 Can you do preprocessing to optimize this solution?
- #715.** 16.11 Once you have a recursive algorithm, think about the runtime. Can you make this faster? How?
- #716.** 16.1 Let `diff` be the difference between a and b . Can you use `diff` in some way? Then can you get rid of this temporary variable?
- #717.** 17.19 Part 2: You might need the quadratic formula. It's not a big deal if you don't remember it. Most people won't. Remember that there is such a thing as good enough.
- #718.** 16.18 Since the value of a determines the value of b (and vice versa) and either a or b must start at the beginning of the value, you should have only $O(n)$ possibilities for how to split up the pattern.
- #719.** 17.12 You could return both the start and end of a linked list in multiple ways. You could return a two-element array. You could define a new data structure to hold the start and end. You could re-use the `BiNode` data structure. If you're working in a language that supports this (like Python), you could just return multiple values. You could solve the problem as a circular linked list, with the start's previous pointer pointing to the end (and then break the circular list in a wrapper method). Explore these solutions. Which one do you like most and why?

IV | Hints for Additional Review Problems

- #720.** 16.23 You can implement `rand7()` with `rand5()`, you just can't do it deterministically (such that you know it will definitely terminate after a certain number of calls). Given this, write a solution that works.
- #721.** 17.23 You should be able to do this in $O(N^3)$ time, where N is the length of one dimension of the square.
- #722.** 16.11 Consider memoization to optimize the runtime. Think carefully about what exactly you cache. What is the runtime? The runtime is closely related to the max size of the table.
- #723.** 16.19 You should have an algorithm that's $O(N^2)$ on an $N \times N$ matrix. If your algorithm isn't, consider if you've miscomputed the runtime or if your algorithm is suboptimal.
- #724.** 17.1 You might need to do the add/carry operation more than once. Adding carry to sum might cause new values to carry.
- #725.** 17.18 Once you have the precomputation solution figured out, think about how you can reduce the space complexity. You should be able to get it down to $O(SB)$ time and $O(B)$ space (where B is the size of the larger array and S is the size of the smaller array).
- #726.** 16.20 We're probably going to run this algorithm many times. If we did more preprocessing, is there a way we could optimize this?
- #727.** 16.18 You should be able to have an $O(n^2)$ algorithm.
- #728.** 16.7 Have you considered how to handle integer overflow in $a - b$?
- #729.** 16.5 Each factor of 10 in $n!$ means $n!$ is divisible by 5 and 2.
- #730.** 16.15 For ease and clarity in implementation, you might want to use other methods and classes.
- #731.** 17.18 Another way to think about it is this: Imagine you had a list of the indices where each item appeared. Could you find the first possible subsequence with all the elements? Could you find the second?
- #732.** 16.4 If you were designing this for an $N \times N$ board, how might your solution change?
- #733.** 16.5 Can you count the number of factors of 5 and 2? Do you need to count both?
- #734.** 17.21 Each bar will have water on top of it that matches the minimum of the tallest bar on the left and the tallest bar on the right. That is, `water_on_top[i] = min(tallest_bar(0->i), tallest_bar(i, n))`.
- #735.** 16.16 Can you expand the middle until the earlier condition is met?
- #736.** 17.23 When you're checking to see if a particular square is valid (all black borders), you check how many black pixels are above (or below) a coordinate and to the left (or right) of this coordinate. Can you precompute the number of black pixels above and to the left of a given cell?
- #737.** 16.1 You could also try using XOR.
- #738.** 17.22 What if you did a breadth-first search starting from both the source word and the destination word?
- #739.** 17.13 In real life, we would know that some paths will not lead to a word. For example, there are no words that start with `hellothisism`. Can we terminate early when going down a path that we know won't work?

- #740.** 16.11 There's an alternate, clever (and very fast) solution. You can actually do this in linear time without recursion. How?
- #741.** 17.18 Consider using a heap.
- #742.** 17.21 You should be able to solve this in $O(N)$ time and $O(N)$ space.
- #743.** 17.17 Alternatively, you could insert each of the smaller strings into the trie. How would this help you solve the problem? What is the runtime?
- #744.** 16.20 With preprocessing, we can actually get the lookup time down to $O(1)$.
- #745.** 16.5 Have you considered that 25 actually accounts for two factors of 5?
- #746.** 16.16 You should be able to solve this in $O(N)$ time.
- #747.** 16.11 Think about it this way. You are picking K planks and there are two different types. All choices with 10 of the first type and 4 of the second type will have the same sum. Can you just iterate through all possible choices?
- #748.** 17.25 Can you use a trie to terminate early when a rectangle looks invalid?
- #749.** 17.13 For early termination, try a trie.

XIV

About the Author

Gayle Laakmann McDowell has a strong background in software development with extensive experience on both sides of the hiring table.

She has worked for Microsoft, Apple, and Google as a software engineer. She spent three years at Google, where she was one of the top interviewers and served on the hiring committee. She interviewed hundreds of candidates in the U.S. and abroad, assessed thousands of candidate interview packets for the hiring committee, and reviewed many more resumes.

As a candidate, she interviewed with—and received offers from—twelve tech companies, including Microsoft, Google, Amazon, IBM, and Apple.

Gayle founded CareerCup to enable candidates to perform at their best during these challenging interviews. CareerCup.com offers a database of thousands of interview questions from major companies and a forum for interview advice.

In addition to *Cracking the Coding Interview*, Gayle has written other two books:

- ***Cracking the Tech Career: Insider Advice on Landing a Job at Google, Microsoft, Apple, or Any Top Tech Company*** provides a broader look at the interview process for major tech companies. It offers insight into how anyone, from college freshmen to marketing professionals, can position themselves for a career at one of these companies.
- ***Cracking the PM Interview: How to Land a Product Manager Job in Technology*** focuses on product management roles at startups and big tech companies. It offers strategies to break into these roles and teaches job seekers how to prepare for PM interviews.

Through her role with CareerCup, she consults with tech companies on their hiring process, leads technical interview training workshops, and coaches engineers at startups for acquisition interviews.

She holds bachelor's degree and master's degrees in computer science from the University of Pennsylvania and an MBA from the Wharton School.

She lives in Palo Alto, California, with her husband, two sons, dog, and computer science books. She still codes daily.



Amazon.com's #1 Best-Selling Interview Book

CRACKING *the* CODING INTERVIEW

I am not a recruiter. I am a software engineer. And as such, I know what it's like to be asked to whip up brilliant algorithms on the spot and then write flawless code on a whiteboard. I've been through this—as a candidate and as an interviewer.

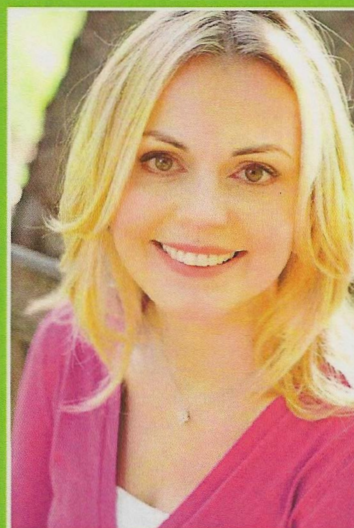
Cracking the Coding Interview, 6th Edition is here to help you through this process, teaching you what you need to know and enabling you to perform at your very best. I've coached and interviewed hundreds of software engineers. The result is this book.

Learn how to uncover the hints and hidden details in a question, discover how to break down a problem into manageable chunks, develop techniques to unstuck yourself when stuck, learn (or re-learn) core computer science concepts, and practice on 189 interview questions and solutions.

These interview questions are real; they are not pulled out of computer science textbooks. They reflect what's truly being asked at the top companies, so that you can be as prepared as possible.

WHAT'S INSIDE?

- 189 programming interview questions, ranging from the basics to the trickiest algorithm problems.
- A walk-through of how to derive each solution, so that you can learn how to get there yourself.
- Hints on how to solve each of the 189 questions, just like what you would get in a real interview.
- Five proven strategies to tackle algorithm questions, so that you can solve questions you haven't seen.
- Extensive coverage of essential topics, such as big O time, data structures, and core algorithms.
- A "behind the scenes" look at how top companies, like Google and Facebook, hire developers.
- Techniques to prepare for and ace the "soft" side of the interview: behavioral questions.
- For interviewers and companies: details on what makes a good interview question and hiring process.



GAYLE LAAKMANN MCDOWELL

Gayle Laakmann McDowell is the founder and CEO of CareerCup and the author of *Cracking the PM Interview* and *Cracking the Tech Career*.

Gayle has a strong background in software development, having worked as a software engineer at Google, Microsoft, and Apple. At Google, she interviewed hundreds of software engineers and evaluated thousands of hiring packets as part of the hiring committee. She holds a B.S.E. and M.S.E. in computer science from the University of Pennsylvania and an MBA from the Wharton School.

She now consults with tech companies to improve their hiring process and with startups to prepare them for acquisition interviews.

6TH
EDITION

ISBN 9780984782857



9 780984 782857

90000 >

